

Pointers and Structures

Pointers and Structures

- **Structure pointers are just like pointers to a variable of any data-type.**

Declaring a structure pointer

```
struct <struct-name> *<pointer name>;
```

For eg.,

```
struct dob
{
    int day;
    int month;
    int year;
};
struct dob *person1;
```

This declaration for a pointer to structure does not allocate any memory for a structure but allocates only for a pointer, so that to access structure's members through pointer person1, we must allocate the memory using ***malloc()*** function.

Access Structure Members - Pointers

- Just like we use "." operator to access members of a structure using a normal variable, while for a structure pointer we use "->" notation.
- For eg.,

```
person1->date;  
person1->month;  
person1->year;
```

- Other method to access structure members through structure pointers.

```
(*person1).date;  
(*person1).month;  
(*person1).year;
```

```
*person1.date;  
*person1.month;  
*person1.year;
```

Access Structure Members - Pointers

- When using structure pointers, we should take care of operator precedence.
 - Member operator “.” has higher precedence than “*”.
 - **person1 -> date** and **(*person1).date** mean the same thing.
 - ***person.date** will lead to error.
 - The operator “->” enjoys the highest priority among operators.
 - **++person1 -> date** will increment **date**, not **person1**.
 - **(++person1) -> date** will do the intended thing.
- When pointer is incremented by one i.e., **++person1**, it increases by **sizeof(dob)**, and points the next record, if any.

Example Program

The address of a structure type variable can be stored in a structure type pointer variable

```
#include<stdio.h>

struct point
{
    int x, y;
};

int main()
{
    struct point p1 = {1, 2};

    // p2 is a pointer to structure p1
    struct point *p2 = &p1;

    // Accessing structure members using structure pointer
    printf("%d %d", p2->x, p2->y);
    return 0;
}
```

Example Program using malloc()

```
#include<stdio.h>
struct point
{
    int x, y;
};
int main()
{
    struct point *p1;
    p1=(struct point *)malloc(sizeof(struct point)); //dynamic memory allocation
    printf("Enter x,y coordinates");
    scanf("%d%d",&p2->x,&p2->y);
    printf("%d %d", p2->x, p2->y); // Accessing structure members
    return 0;
}
```

malloc() will be covered in detail in future lecture

References

- https://cse.iitkgp.ac.in/~bivasm/pds_notes/
- <https://www.geeksforgeeks.org/>
- <https://overiq.com/c-programming-101/>
- <https://www.oreilly.com/library/view/understanding-and-using/9781449344535/ch04.html>
- <https://www.javatpoint.com/pointer-arithmetic-in-c>
- https://www.cs.swarthmore.edu/~newhall/cs31/resources/C-structs_pointers.php
- <http://www2.ece.ohio-state.edu/~degroat/EG167C/Lecture%2014%20-%20Pointers%20-%2006.ppt>

End of Lecture